Noise, Coding and Receiver Design for a Screen-to-Camera Optical Link

Riyaadh Gani, Omar Sayed, Adrian Pailler, and Lara Yanc
University College London
COMP0228: Sampling, Compression and Data Analysis

***Abstract*—We investigate how reliably text can be sent using binary light signalling. A patch on a Mac screen displays white for one and black for zero; a phone records a video of these changes. Software reads the brightness to recover the bits and reconstruct the text. We disturb reception by shining light onto the screen or blocking the camera's view. Across 21 videos, we test three ways of adding extra bits to help repair errors: repetition, Hamming and Reed–Solomon coding. We measure message recovery and transmission time. We also test whether measuring brightness differently or rearranging bits before transmission improves recovery. The receiver restores the original bit order before decoding. Brightness methods process the same videos, so each faces the same recorded interference. Measuring a larger screen area raises observed recovery from 8/12 to 12/12 messages in the main glare tests. Interleaved Hamming has higher observed recovery than its non-interleaved counterpart, whereas interleaved Reed–Solomon has lower recovery in the recorded blockage trials. Because these transmissions experienced different disturbances, controlled simulations complement the physical results by examining how error grouping affects correction. These findings show that adding protection alone is insufficient: its effectiveness depends on the errors encountered. Small samples and manually varied disturbances limit wider conclusions.**

***Index Terms*—optical communication, error correction, interleaving, sampling, experimental design**

I. Introduction

We investigate reliable text transmission using binary light signalling. A Mac screen shows each message as white and black flashes representing ones and zeros. A phone records these changes as a video, which software reads to reconstruct the text. This screen–camera approach, also explored in PixNet [1], allows us to observe how glare and blockage disturb reception.

These disturbances can change the brightness readings, causing errors in the recovered text. Error-correcting codes add extra bits to repair some mistakes, but take longer to transmit. We therefore examine which errors occur, whether the codes address them, and what further improvements are needed.

We first explain the communication system (Section II) and experimental design (Section III), then compare the codes through their message recovery and transmission time (Section IV). The observed failures motivate two extensions: improving how brightness is measured (Section V), and rearranging bits during transmission to spread consecutive errors before correction (Section VI). We evaluate brightness methods on the same videos and use controlled computer simulations to complement physical trials of coding and bit order. We conclude by assessing the findings and limitations of the evidence (Section VII).

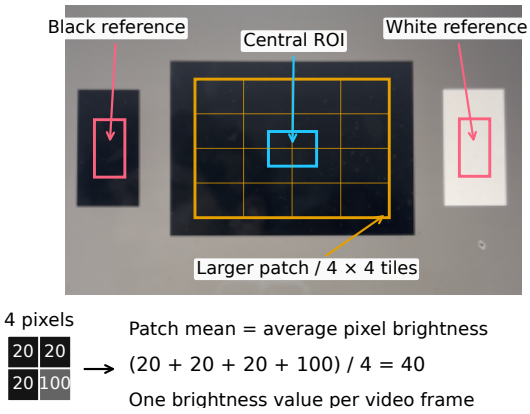


Fig. 1. Top: recorded screen with central ROI (blue), larger patch and tiles (orange), and references (pink, each split in two). Bottom: illustrative four-pixel average; actual regions contain many pixels. Both central and larger-patch means give one brightness value per frame.

II. Building and Understanding the Link

A. Sending Text as Light

The transmitter converts text into UTF-8 bytes, each containing eight bits. A Python program displays these bits as white or black, holding each for 200 ms. The phone records the screen at approximately 30 frames/s, giving about six images per bit. Recorded timestamps locate measurements despite uneven frame spacing.

To distinguish messages, we organise each transmission into a *packet*:

8 guard bits → 32-bit preamble → 48-bit header → coded message and CRC → 8 guard bits.

The *preamble* is a known pattern for locating the message and measuring dark and bright levels. The header identifies the code, length, version and sequence; dark guards separate packets. Header and message each carry a 16-bit cyclic redundancy check (CRC), calculated from their contents to detect changes. Only the message and its CRC receive error correction. When bits are rearranged, both ends use the same predefined order; the header does not record it.

We use 1920×1080 iPhone 15 and 15 Pro videos.

B. Reading Bits from Video

We average grayscale pixels inside the blue region in Fig. 1, turning each image into a brightness measurement. Its width and height are about 20% of the visible patch, chosen from unobstructed frames without testing which region decodes best. The receiver finds the preamble and uses the midpoint between its dark and bright levels as a threshold. For each

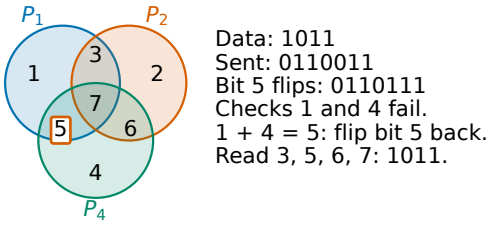


Fig. 2. Verified Hamming example. Circles show which positions each parity check covers; position 5 belongs to checks 1 and 4.

bit, it takes the median brightness within the middle half of the 200 ms interval, avoiding many display transitions. Values above the threshold become ones; the rest become zeros.

The receiver estimates packet start from the known preamble but fixes bit duration at the transmitter's configured 200 ms. Estimating duration from the preamble caused later sampling intervals to drift out of alignment in pilot trials. The fixed-duration approach recovered 16/16 pilot messages without errors in 3,840 body bits. This development result guided main analysis; it is not independent validation. The header must pass its CRC before the message is decoded and checked. Expected message text is never supplied to the receiver or used to select timing.

C. Correcting Errors and Paying for Redundancy

Without coding, each message bit is sent once. **Repetition** sends three or five copies and takes their majority: `000` received as `010` still gives zero, but `110` does not. Three copies tolerate one error per group; five tolerate two.

Hamming$(7, 4)$ adds three parity bits to four data bits. Each parity check requires an even number of ones in an overlapping set of positions [2]. Together, the failed checks give a *syndrome* that locates one wrong bit (Fig. 2). Flipping it repairs that word; two errors can instead identify the wrong position.

Reed–Solomon instead protects bytes. The credited `reedsolo` library adds four parity bytes and corrects up to two damaged bytes [3], [4]. Several wrong bits within one byte count as one damaged byte; one wrong bit in each of three bytes exceeds the limit. After correction, CRC detects many remaining errors but neither repairs them nor guarantees the original message. Including CRC and framing, an eight-byte message takes 99.2 s with five-copy repetition versus 47.2 s with Hamming. Better recovery must therefore justify the added transmission time.

III. Experimental Design and Evidence

A. Stages, Controls and Measurement Units

Table I separates development, code evaluation and acquisition stress across 21 videos. Payloads are four, eight and twelve bytes: `A7k2`, `A7k2M9q4`, and `A7k2M9q4R6v8`. They standardise content within each length and include mixed character patterns. Because each length has only one message and the strings share a prefix, length and content effects cannot be separated.

Main blockage testing uses seven configurations: uncoded, repetition $\times 3$, repetition $\times 5$, Hamming and Reed–Solomon without interleaving (depth one), plus Hamming and Reed–Solomon at depth eight. Each occurs at three lengths in two rounds, giving six packets per configuration. The ordinary Hamming/Reed–Solomon controls are reused for the interleaving comparison; they are not counted twice. Body-glare testing compares uncoded and Hamming at each length and round. Acquisition-stress tests instead apply early glare to uncoded, Hamming and Reed–Solomon transmissions of the long message.

B. Experimental Controls

Transmitter logs verify each packet's message, code, interleaving depth, order and timing. Receiver settings remain fixed during main analysis, and all seven processing methods use the same 42 blockage and 12 glare packets. Each method therefore encounters identical recorded interference.

Physical interventions target 1.2 s, but manual duration, coverage and intensity vary. Comparisons between separately disturbed coded packets therefore describe performance under the recorded conditions; receiver comparisons on the same videos provide stronger control. Failures and packets with no observed errors remain in the analysis.

C. Outcomes and Statistical Scope

Success requires both exact payload agreement and a valid CRC. Failures are retained by stage: acquisition/header, sampling, error correction or payload CRC. Body bit-error rate (BER) is the fraction of received body bits that differ from those sent, before correction. Missing bodies are unavailable, not zero BER.

To account for the cost of redundancy, we also measure goodput: correctly delivered payload bits per second of transmission. For n_i coded-body bits and b_i payload bits, packet duration T_i and overall goodput G are

$$T_i = 0.2(96 + n_i), \qquad G = \frac{\sum_i b_i \, \mathbf{1}(\text{exact+CRC})}{\sum_i T_i}. \tag{1}$$

The indicator is one only for exact recovery with a valid CRC. Every included packet contributes its transmission duration to the denominator, including packets that fail. Transmission time includes 96 framing intervals but excludes the 0.2 s lead-in, eight-second gaps, setup and offline processing.

Physical results show counts and individual outcomes: bits within a packet and packets sharing a video are not independent observations. Two rounds provide limited repetition, so we avoid significance tests or narrow intervals for physical code rankings. Simulation intervals instead quantify Monte Carlo uncertainty under the specified noise model.

IV. Physical Results: Errors and Coding

A. Observed Errors and Detection Failures

The clean pilot recovers 16/16 messages without errors in 3,840 body bits. Under the main disturbances, the receiver detects all 54 packets but recovers only 35 messages: the failures occur after detection. Clean reception establishes that the link can work.

Glare produces 18/1,320 wrong bits (1.36%), all $0 \to 1$: dark symbols are read as bright. Blockage produces 78/7,344 (1.06%), with 34 $0 \to 1$ and 44 $1 \to 0$ errors. Rejecting unusually bright readings may therefore address glare, but cannot address both directions of blockage error.

TABLE I
EXPERIMENTAL QUESTIONS, COMPARISONS AND PACKET COUNTS. EACH TRANSMISSION IS COUNTED ONCE; PROCESSING THE SAME VIDEO AGAIN ADDS NO PACKETS.

Short: `A7k2` (4 bytes) Medium: `A7k2M9q4` (8 bytes) Long: `A7k2M9q4R6v8` (12 bytes)

Question	Experiment	Packets	Why this comparison is useful
1. Does the clean link work?	Short/long × uncoded/repetition ×5 × interleaving off/on × two repeats	16	Checks timing and decoding at two message lengths, with repeated clean trials.
2. How do disturbances affect reception?	Medium message × three disturbances × uncoded/Hamming	6	Keeps the message fixed while comparing disturbances; calibration stays separate from evaluation.
3. What does coding contribute?	Three lengths × five codes × two rounds, under blockage	30	Gives every code the same messages and repetition count; compares recovery alongside transmission time.
4. What does interleaving contribute?	Three lengths × interleaved Hamming/Reed–Solomon × two rounds	12 extra	Reuses non-interleaved controls from row 3; tests bit order without duplicating controls.
5. Does processing help without coding?	Three lengths × uncoded glare recordings × two rounds	6	Each method processes identical frames, testing brightness processing without error correction.
6. Does processing help alongside coding?	Three lengths × Hamming glare recordings × two rounds	6	Tests processing followed by error correction on identical recorded inputs.
7. Where does message detection fail?	Long message × three codes × two repeats, with early glare	6	Keeps message content fixed while testing detection of the starting pattern, including failures.
Total	**21 videos**	**82**	

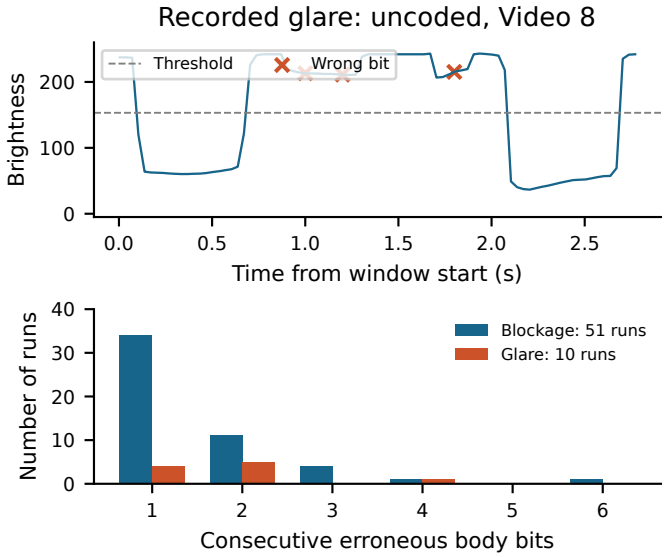


Fig. 3. Top: example glare recording and wrong-bit centres. Bottom: 51 blockage and ten glare error runs across 54 main packets; packet boundaries terminate runs. Runs do not measure disturbance duration: a disturbed bit may remain correct. Zero-error packets remain in delivery results.

Runs of at least two consecutive wrong bits account for 44/78 blockage errors (56%) and 14/18 glare errors (78%) in Fig. 3. Most errors therefore occur in clusters, particularly under glare. Several errors within one protected group can exceed its correction limit even when overall BER is low. This motivates interleaving, which changes how consecutive errors reach those groups.

Body-glare recovery is 8/12, with all four failures in one capture setup and no baseline body errors in the other eight packets. This limits claims of consistent glare robustness. Early glare instead prevents all six packets from yielding valid headers; body BER is unavailable. Here, recovery requires protecting the starting pattern or header before body correction can help.

TABLE II
BLOCKAGE RESULTS: SIX PACKETS PER CONFIGURATION. PRE-CORRECTION BER; GOODPUT INCLUDES FAILURES.

Code/depth	Success	Errors/bits	BER (%)	bit/s
Uncoded / 1	1/6	6/480	1.25	0.152
Repetition ×3 / 1	3/6	9/1440	0.62	0.476
Repetition ×5 / 1	5/6	10/2400	0.42	0.538
Hamming / 1	4/6	9/840	1.07	1.130
Hamming / 8	6/6	11/840	1.31	1.356
Reed–Solomon / 1	6/6	12/672	1.79	1.538
Reed–Solomon / 8	2/6	21/672	3.12	0.513

B. Message Recovery and Transmission Cost

Table II compares the seven blockage configurations, with individual outcomes in Fig. 4. Repetition ×5 recovers 5/6 messages versus 3/6 for ×3, but goodput rises only from 0.476 to 0.538 bit/s because the extra copies take longer to send. Reed–Solomon without interleaving recovers 6/6 and achieves the highest observed goodput, 1.538 bit/s. Recorded recovery is higher for interleaved Hamming and lower for interleaved Reed–Solomon than for their respective non-interleaved configurations. These separately disturbed transmissions do not isolate the effect of interleaving; Section VI investigates this contrast.

Repetition ×5 has lower BER than non-interleaved Reed–Solomon (0.42% versus 1.79%), yet lower recovery and goodput. Extra copies can dilute one disturbance's errors across more transmitted bits. Lower BER alone therefore does not establish better communication; message recovery and transmission cost must also be considered.

Recovery falls from 35/54 to 30/54 when using the earliest available completed recordings. This sensitivity to recording selection limits claims about reliability in new conditions.

V. EXTENSION 1: RECEIVER PROCESSING

A. Receiver Comparisons

We compare seven methods on the same frames. The screen displays one bit across the whole transmitting patch, but local glare can dominate a small central region of interest (ROI).

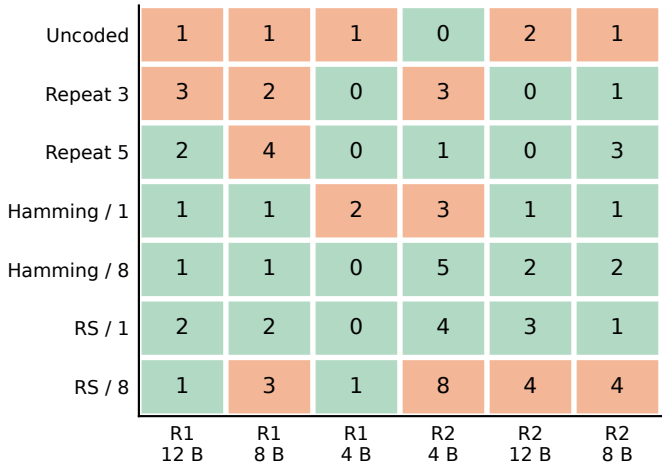


Fig. 4. All 42 blockage packets by round and message length. Green/orange denotes exact+CRC success/failure; numbers give pre-correction bit errors.

A larger-patch mean averages brightness over a wider area, allowing unaffected pixels to dilute that glare (Fig. 1). It cannot remove glare covering the entire region. We use the inner 80% of the patch's width and height, or a fourfold enlargement of the central box where full bounds are unavailable.

Reference compensation addresses lighting changes shared across the image. Because the reference strips do not carry the changing message, their brightness changes can estimate this disturbance. We subtract the median change across four reference regions, relative to their preamble levels, from the larger-patch mean. This relies on similar illumination changes at the references and signal; obstructed references can mislead it.

Spatial processing addresses glare concentrated in part of the patch. We divide the region into 4×4 tiles, normalise each using its preamble dark and bright levels, and take the lower quartile of valid readings. This favours darker tiles that glare may have spared, rather than letting bright tiles pull up the average. At least half the tiles must have 20 grayscale units of contrast. Dark blockage can bias this choice; the combined method applies reference compensation before spatial processing.

Temporal filtering smooths the larger-patch mean, using the expectation that brightness remains steady within each bit. The exponentially weighted moving average (EWMA) blends successive measurements; Kalman filtering weights the previous estimate and new measurement using estimated uncertainty [6]. Smoothing may suppress brief fluctuations, but must preserve genuine bit transitions. Both use actual frame intervals Δt_i: EWMA uses $a_i = 1 - \exp(-\Delta t_i / 0.025)$; Kalman process variance is $160000 \Delta t_i$, with measurement and initial variance 400. Their new-measurement weights at 30 frames/s are approximately 0.74 and 0.93, favouring responsiveness over strong smoothing. Neither can recover a signal completely hidden by sustained blockage.

Settings were fixed using development data. All methods share baseline packet detection and timing, then calibrate their brightness thresholds from the known preamble. Expected message text is never supplied to the decoder, so the comparison tests measurement changes after detection.

TABLE III
RECEIVER ABLATIONS ON THE SAME MAIN RECORDINGS. ENTRIES ARE EXACT+CRC PACKETS AND RAW BODY ERRORS. REFERENCE-BASED BLOCKAGE RESULTS DELIBERATELY INCLUDE REFERENCE OCCLUSION. ALL METHODS ALSO RETAIN 0/6 ACQUISITION-STRESS RECOVERY THROUGH SHARED ACQUISITION.

	Body glare		Blockage	
Method	Success	Errors	Success	Errors
Central ROI	8/12	18	27/42	78
Larger-patch mean	12/12	0	29/42	75
Reference	12/12	0	33/42	72
Spatial quartile	12/12	0	29/42	78
Combined	12/12	0	33/42	73
EWMA	12/12	0	30/42	71
Kalman	12/12	0	29/42	75

B. *Paired Results*

Enlarging the measured region raises body-glare recovery from 8/12 to 12/12 and removes all 18 raw errors (Table III). Every additional method also reaches 12/12. The improvement is therefore already explained by measuring a larger area; these recordings show no further benefit from filtering or reference compensation. In calibration, spatial and combined processing recovered 2/2 glare packets where the larger mean recovered 0/2. That motivated testing, but its advantage did not carry over to the main glare comparison.

Under blockage, the larger mean raises recovery from 27/42 to 29/42, rescuing three messages while losing one previously recovered. EWMA gives 30/42 and Kalman 29/42. Reference and combined processing reach 33/42, each rescuing six messages without losing any baseline successes. However, some blockages cover the reference regions, so these improvements do not establish that compensation correctly estimated illumination changes. EWMA has the fewest raw errors (71), yet recovers fewer messages than the reference method (72 errors). A lower error count therefore does not guarantee better recovery: correction also depends on how errors are grouped.

The benefit also depends on the code. Combined processing raises uncoded blockage recovery from 1/6 to 2/6 and non-interleaved Hamming from 4/6 to 5/6. Enlarging the region alone instead lowers Hamming to 3/6 despite improving the overall total. Interleaved Hamming and non-interleaved Reed–Solomon remain at 6/6 throughout. A receiver improvement in aggregate therefore need not improve every coding configuration.

These results favour measuring a sufficiently broad screen area before adding more processing. Temporal filtering offers little additional benefit here, while reference compensation depends on suitable reference signals. All seven methods retain the six early-glare failures because they share packet detection; this experiment cannot establish improved acquisition.

VI. EXTENSION 2: INTERLEAVING AND CONTROLLED NOISE

A. *Reordering Bits Before Correction*

Suppose a short blockage causes three consecutive bit errors. If all three fall in one Hamming word, they exceed its one-error correction limit. In a simple three-word example, interleaving sends one bit from word A, then B, then C, repeating this order. After the receiver restores the original

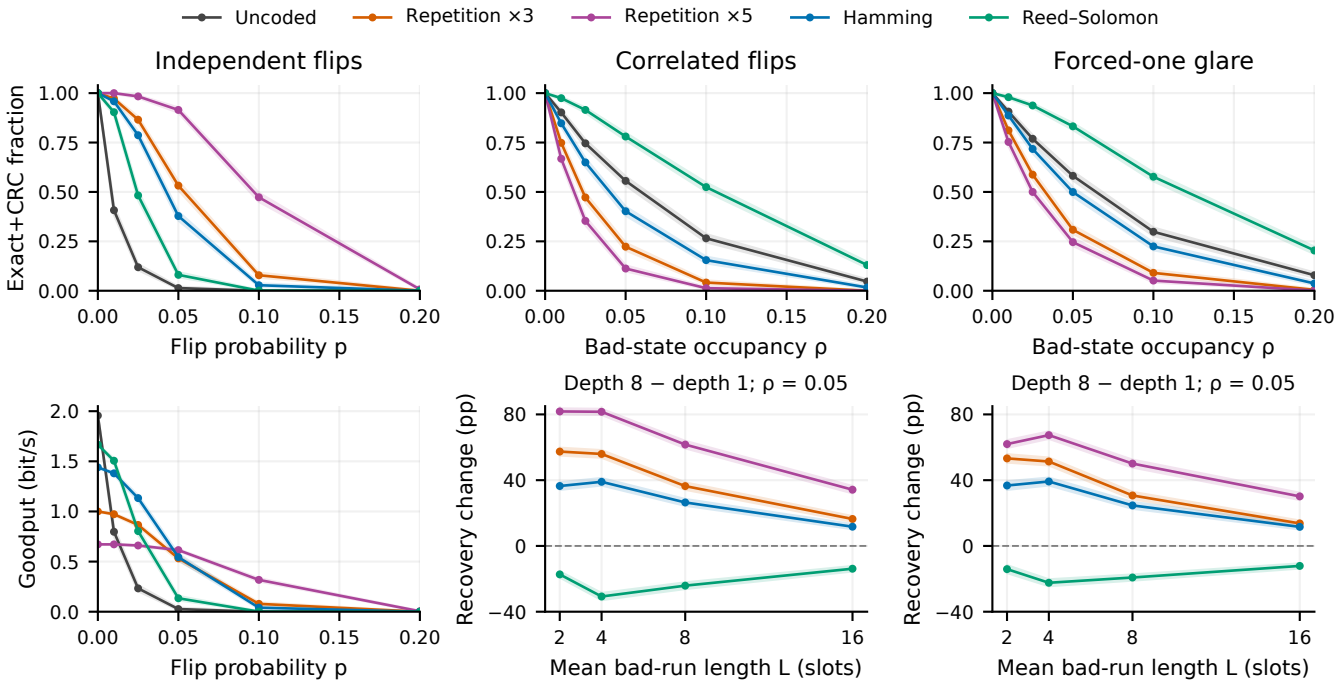


Fig. 5. Saved simulations, 2,000 paired trials per condition and nine-byte payloads. Top: depth-one recovery versus noise rate ($L = 8$ for bursts). Bottom left: independent-flip goodput. Those bands are 95% Wilson intervals [5]. Bottom middle/right: depth-eight minus depth-one recovery in percentage points versus L, at fixed occupancy $\rho = 0.05$, for the four coded methods. Bands are pointwise 95% paired percentile-bootstrap intervals (20,000 resamples). Glare occupancy is not BER. All intervals describe model uncertainty, not physical-scene reliability.

TABLE IV
RECORDED DEPTH-EIGHT EXAMPLES FROM VIDEO 19. ALL INDICES START AT ZERO; PACKETS EXPERIENCED SEPARATE DISTURBANCES.

Zero-based indices	Hamming	Reed–Solomon
Wire bit	50, 52	43, 44, 45, 47
After inverse	42, 78	47, 61, 75, 103
Word / byte	6, 11	5, 7, 9, 12
Correction budget	1 bit/word	2 bytes/body
Observed outcome	Pass	Fail

order, each word has one error and all three can be repaired. Interleaving has not removed errors; it has distributed them into groups the code can correct.

Our implementation writes coded bits across eight rows and transmits them down columns, skipping empty positions. The receiver reverses this order before correction. No bits or transmission airtime are added; buffering latency is unmeasured. Reed–Solomon illustrates why this is not always helpful: it corrects two damaged bytes across the body. Several wrong bits within one byte still count as one damaged byte, but spreading them into three bytes exceeds its correction limit.

Physical Hamming recovery rises from 4/6 to 6/6 with interleaving, although raw errors rise from nine to eleven. Reed–Solomon falls from 6/6 to 2/6: its four interleaved failures damage 3, 7, 3 and 4 bytes. All non-interleaved packets damage at most two bytes, including one with four wrong bits. Table IV shows two recorded examples: separate Hamming words each remain correctable, whereas four damaged Reed–Solomon bytes exceed the budget.

B. Controlled Noise Models

Physical disturbances vary between transmissions. To compare coding and bit order under shared noise, we use 480,000 simulated trials: 240 conditions with 2,000 trials each. Comparisons share nine-byte payloads and noise sequences; longer coded bodies encounter more of each sequence. Saved seeds reproduce the trials; 720 decodes were replayed during verification.

Independent noise flips each bit with probability p. To represent clusters, a two-state model alternates between good and bad periods [7]. The long-run fraction of bad slots is ρ, and their mean consecutive length is L bit intervals. Its transition probabilities are

$$P(B \to G) = 1/L, \qquad P(G \to B) = \rho/[(1-\rho)L]. \quad (2)$$

Good slots leave bits unchanged. Bad slots either flip bits or force them to one, representing the direction observed under glare. Forced-one noise leaves transmitted ones correct, so ρ is not its BER. Initial states use the stationary probabilities. We sweep p or ρ through $0, 0.01, 0.025, 0.05, 0.10, 0.20$, with $L = 8$ for bursts, then test $L = 2, 4, 8, 16$ at $\rho = 0.05$. Only the coded body is disturbed; packet detection and headers are ideal, but framing airtime is included. These models test error mechanisms rather than reproduce a particular camera.

C. Simulation Results and Design Implications

Figure 5 shows that interleaving's benefit depends on both code and burst structure. Under correlated flips at $\rho = 0.05$, Hamming gains 39.0 percentage points at $L = 4$ (95% paired interval: 36.9–41.2), but 11.7 at $L = 16$ (10.4–13.2). Reed–Solomon loses 30.75 and 13.85 points, respectively, and is

harmed throughout the tested length range under both burst models. The practical choice is therefore the code and bit order together: spreading errors helps Hamming when it keeps each word correctable, but can harm Reed–Solomon by damaging more bytes. The smaller Hamming gain at $L = 16$ also shows that the same interleaver does not provide equal protection across burst lengths.

At fixed ρ, longer bad periods also occur less frequently, allowing some short packets to avoid them entirely. The length sweep therefore changes both the severity of an encounter and its likelihood. The plotted intervals resample matched trial pairs and quantify uncertainty within each model condition; more simulated trials do not provide more physical recording conditions.

At $\rho = 0.05$ and $L = 8$, interleaved repetition $\times 5$ recovers 72.90% of messages under correlated flips, exceeding Hamming's 66.70%. Yet its goodput is only 0.490 bit/s versus 0.960 bit/s: the extra copies improve delivery probability while nearly halving useful delivery rate. Code selection therefore depends on whether reliable individual delivery or useful throughput matters more. One simulated wrong payload passes CRC; it remains a failure because success requires actual message recovery.

VII. Validation, Limitations and Conclusions

A. Verification and Experimental Limits

Saved logs, brightness traces and simulation seeds make the outcomes reproducible. Tests check coding, correction limits, interleaving reversal, and detection and sampling failures. Clean synthetic tests cover every code, depth and receiver combination without supplying expected messages to the decoder. The current full suite passes 124 tests.

The receiver comparisons provide the clearest physical evidence because each method processes identical recorded interference. Separate code transmissions experience manually varied disturbances, while two rounds and different capture setups limit conclusions about general reliability. Section IV also quantifies sensitivity to recording selection. A follow-up should automate blockage and glare, test every condition across setups, and use several messages per length. This would separate coding effects from exposure and message content. Improving packet detection is also necessary: the six early-glare failures occur before body correction can help.

B. Connection to the Module and Wider Practice

Week 1's sampling ideas underpin the camera timing, brightness regions and within-bit median. Measuring a wider region can help when repeated samples of an obstructed region cannot. Week 2's repetition and Hamming codes expose the redundancy–rate trade-off; Reed–Solomon and interleaving extend it to different correction groups.

Weeks 2–3 connect uncertainty to its probability model. Our directional, clustered errors violate the independent, symmetric assumptions of the binary symmetric channel, so average BER alone cannot justify its capacity formula. Lab 2's transmitter–channel–receiver experiment is implemented optically. For robotics, this illustrates why sensor processing should follow an understanding of how measurements fail.

C. Conclusion

We built a screen-to-camera link that transmits text through binary light signals, then used physical recordings and controlled simulations to investigate how glare and blockage affect recovery. The experiments showed that reliable communication depends on where errors arise and how they are grouped, as well as how many occur.

Measuring a larger screen area raised glare recovery from 8/12 to 12/12 without additional filtering. Interleaving helped Hamming by distributing errors across independently correctable words, but harmed the tested Reed–Solomon configuration by spreading damage across too many bytes. These findings show why a more elaborate receiver or additional protection is not automatically better: each change must address the observed failure mechanism and justify its transmission cost.

The clearest next step is to strengthen packet detection, since all six transmissions with glare over the starting pattern failed before body correction could help. Repeated starting patterns and error-protected headers are concrete options to test. Automated disturbances would then allow these changes and coding choices to be compared under repeatable interference. The resulting design should preserve the benefits of broader brightness measurement while selecting coding and bit order together, judging success through both recovered messages and useful bits delivered per second.

Acknowledgment

AI tools assisted implementation, analysis, verification and report drafting.

References

[1] S. D. Perli, N. Ahmed, and D. Katabi, "PixNet: Interference-free wireless links using LCD-camera pairs," in *Proc. ACM MobiCom*, 2010, pp. 137–148. https://people.csail.mit.edu/nabeel/pixnet-mobicom10.pdf.

[2] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, 1950. doi:10.1002/j.1538-7305.1950.tb00463.x.

[3] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *J. Soc. Ind. Appl. Math.*, vol. 8, no. 2, pp. 300–304, 1960. doi:10.1137/0108018.

[4] T. Filiba, S. K. Larroque, and contributors, "reedsolo," version 1.7.0, software. https://pypi.org/project/reedsolo/1.7.0/.

[5] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Stat. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927. doi:10.1080/01621459.1927.10502953.

[6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960. doi:10.1115/1.3662552.

[7] E. N. Gilbert, "Capacity of a burst-noise channel," *Bell Syst. Tech. J.*, vol. 39, no. 5, pp. 1253–1265, 1960. doi:10.1002/j.1538-7305.1960.tb03959.x.